

STARZ

SNEAKER & STREETWEAR E-COMMERCE SPA

CS3404: GUI Programming — Mini Project

The Modern Single Page Application

Computer Engineering

Student: st4rk-7 | Deployed: Netlify | Date: March 2026

1. Features Implemented

Mandatory Requirements

Requirement	Implementation	Status
Tech Stack: Vue 3, TypeScript, Vite, Tailwind CSS	Composition API with <code><script setup lang="ts"></code> , Vite dev server, Tailwind v3.4.17 with custom config	✓
No any Types	Strict interfaces defined: <code>Product</code> , <code>CartItem</code> , <code>AuthUser</code> , <code>LoginResponse</code> . Typed props via <code>defineProps<{}>()</code> and emits via <code>defineEmits<{}>()</code>	✓
Component Architecture	13 reusable components + 7 views (see Section 2 diagram)	✓
Responsive Layout	Mobile hamburger menu, 2-col grid on mobile, 4-col on desktop, sticky pill navbar adapts to all breakpoints	✓
Fetch from API	<code>useProducts</code> composable fetches from <code>/products/category/mens-shoes</code> and <code>/products/category/womens-shoes</code> via Axios	✓

Search / Filter	Real-time text search + 5-facet filter panel (Size, Color, Price, Product Type, Material) + 8 sorting options	✓
Detail View	Dynamic route <code>/product/:id</code> with product gallery, size selector, color swatches, and accordion details	✓

Bonus Features (A/A+ Grade)

Bonus Feature	Implementation	Status
AUTH Authentication Simulation	Pinia <code>useAuthStore</code> calls <code>/auth/login</code> , stores JWT + user data in <code>localStorage</code> , manages login/logout state in <code>NavBar</code> with user dropdown	✓
CART Shopping Cart (Pinia)	Global <code>useCartStore</code> with add/remove/update-quantity actions, computed totals, slide-in <code>CartDrawer</code> with transitions, quick-add from product grid	✓
ROUTER Dynamic Routing	Vue Router with 7 lazy-loaded views: <code>/</code> , <code>/shop</code> , <code>/product/:id</code> , <code>/login</code> , <code>/checkout</code> , <code>/order-confirmation</code> , and a catch-all 404 route	✓
DARK Dark Mode	<code>useTheme</code> composable toggles <code>dark</code> class on <code><html></code> , persists preference to <code>localStorage</code> , respects system <code>prefers-color-scheme</code>	✓

Additional Features (Beyond Requirements)

- **Landing Page:** Five-section homepage with a video hero carousel, category row, featured products, new arrivals, and collection promo tiles
- **Validated Demo Checkout:** Required delivery/payment-format fields, working discount codes, optional returns protection, and a session-persistent order confirmation
- **Mega Menu:** Desktop hover-activated dropdown with category links and image cards
- **Quick Add:** Size selector overlay on product card hover for instant cart additions
- **Currency Localization:** `useCurrency` composable converts USD → LKR with `Intl.NumberFormat`
- **Add-to-Cart Micro-animation:** Three-phase button state (idle → loading spinner → checkmark tick)

2. Architecture & Component Hierarchy

Project Structure

```
src/
├── components/           # 13 reusable UI components
│   ├── NavBar.vue       - Sticky pill navbar, mega menu, mobile drawer, s
│   ├── CartDrawer.vue   - Slide-in cart panel with quantity controls
│   ├── ProductCard.vue  - Product grid item with quick-add overlay
│   ├── ProductGrid.vue  - Responsive grid wrapper
│   ├── ProductGallery.vue - Image gallery for product detail
│   ├── ProductInfo.vue  - Product details, size picker, add-to-cart
│   ├── SidebarFilter.vue - 5-column filter panel (size/color/price/type/ma
│   ├── HeroCarousel.vue - Video hero with controls and reduced-motion sup
│   ├── FeaturedCarousel.vue - Product spotlight carousel
│   ├── CategoryRow.vue  - Category navigation cards
│   ├── NewArrivalsSection.vue - Men's + Women's arrivals grid
│   ├── CollectionPromoTiles.vue - Promotional image tiles
│   └── SiteFooter.vue   - Multi-column footer with links
├── views/               # 7 page-level view components
│   ├── LandingView.vue  - Homepage (5 sections)
│   ├── HomeView.vue     - Shop page with filters + grid
│   ├── ProductDetail.vue - Single product view
│   ├── LoginView.vue    - Authentication form
│   ├── CheckoutView.vue - Full checkout page
│   ├── OrderConfirmationView.vue - Post-purchase confirmation
│   └── NotFoundView.vue - Catch-all 404 state
├── stores/              # 4 Pinia global state stores
│   ├── auth.ts          - JWT auth with localStorage persistence
│   ├── cart.ts          - Cart items, totals, drawer state
│   ├── filter.ts        - Active filters, sort option, price ranges
│   └── search.ts        - Search query, search panel toggle
├── composables/         # 3 reusable composition functions
│   ├── useProducts.ts   - Product fetching, transformation, lookup
│   ├── useTheme.ts      - Dark/light mode toggle + persistence
│   └── useCurrency.ts    - USD → LKR conversion + formatting
├── types/               # TypeScript interface definitions
│   ├── product.ts       - Product catalog interface
│   └── order.ts          - Persisted order interface
├── utils/
│   └── bodyScrollLock.ts - Coordinates modal scroll locking
├── router/
│   └── index.ts          - 7 lazy-loaded views with scroll behavior
├── App.vue              - Root layout (NavBar + router-view + Footer)
├── main.ts              - App bootstrap (Pinia + Router)
└── style.css            - Tailwind directives + base resets
```

Component Hierarchy Diagram

```

App.vue
├── NavBar.vue ───────────────── [uses: cartStore, searchStore, a
│   ├── CartDrawer.vue ───────── [uses: cartStore, useCurrency]
│   ├── Search Overlay (inline)
│   └── Mobile Menu (inline)
├── <router-view />
│   ├── LandingView.vue ───────── [uses: useProducts]
│   │   ├── HeroCarousel.vue
│   │   ├── CategoryRow.vue
│   │   ├── FeaturedCarousel.vue ─ [uses: useProducts]
│   │   ├── NewArrivalsSection.vue ─ [uses: useProducts, useCurrency]
│   │   └── CollectionPromoTiles.vue
│   ├── HomeView.vue (Shop) ───── [uses: useProducts, searchStore,
│   │   ├── SidebarFilter.vue ─── [uses: filterStore]
│   │   └── ProductGrid.vue
│   │       └── ProductCard.vue ─ [uses: cartStore, useCurrency] (
│   ├── ProductDetail.vue ───── [uses: useProducts, cartStore, u
│   │   ├── ProductGallery.vue
│   │   ├── ProductInfo.vue ─── [emits: add-to-cart] [uses: useC
│   │   └── ProductGrid.vue (related items)
│   ├── LoginView.vue ───────── [uses: authStore]
│   ├── CheckoutView.vue ───── [uses: cartStore, useCurrency]
│   ├── OrderConfirmationView.vue ─ [uses: sessionStorage, useCurrenc
│   └── NotFoundView.vue
└── SiteFooter.vue

```

Data Flow Pattern

The application follows a **unidirectional data flow**:

1. **API → Composable:** `useProducts` fetches from DummyJSON and transforms data (augmenting with imagery, mock attributes)
2. **Composable → Views:** Views consume reactive refs (`products` , `isLoading` , `error`) returned by composables
3. **Views → Components:** Data flows down via **typed props** (e.g., `:product="product"`)
4. **Components → Stores:** User interactions trigger **Pinia store actions** (e.g., `cartStore.addToCart()`)
5. **Stores → Components:** Reactive getters propagate state changes globally (e.g., `cartStore.totalItems` updates NavBar badge)

3. Technical Decisions

API Data Transformation Strategy

DummyJSON provides limited shoe data (~10 products). The `useProducts` composable applies a **deterministic transformation pipeline**:

- Fetches real products from `/products/category/mens-shoes` and `/products/category/womens-shoes`
- Augments each product with mock attributes (colors, sizes, materials, product types) using a seeded PRNG for consistency
- Clones each product twice with name variations to populate the grid (~30 products total)
- Replaces thumbnails with high-quality custom product imagery

Justification: The raw API remains the data source of truth (IDs, titles, prices, descriptions). The transformation demonstrates TypeScript proficiency and real-world data adaptation patterns — the same approach used in production e-commerce when API data is sparse.

State Management Architecture

Store	Purpose	Persistence
<code>auth.ts</code>	User session, JWT token, login/logout	<code>localStorage</code> (auto-login on refresh)
<code>cart.ts</code>	Cart items, quantities, selected size/color, drawer toggle	<code>localStorage</code>
<code>filter.ts</code>	Active filters (5 categories), sort option	In-memory
<code>search.ts</code>	Search query, search panel visibility	In-memory

Styling System

The Tailwind configuration extends the default theme with a custom **design system**:

- **Colors:** `brand-blue` (`#1a1f2b`), `brand-red` (`#e31b23`), `offwhite` (`#f5f5f0`), `charcoal` (`#212121`)
- **Typography:** DM Sans (body), Playfair Display (headings/serif), DM Mono (accents)
- **Dark Mode:** Implemented via Tailwind's `darkMode: 'class'` strategy with `dark:` variants on every component

- **Custom Properties:** `border-radius: card (0.75rem)`, `letter-spacing: wide-brand (0.1em)`

4. Git Activity Summary

Total Commits before final submission: 34 | **Feature Branches:** 5 | **Commit Convention:** `feat:` / `fix:` / `refactor:`

Branch Strategy

Branch	Purpose
<code>feature/search-filter</code>	Search bar and category filtering implementation
<code>feature/router</code>	Vue Router setup and dynamic product detail page
<code>feature/cart-logic</code>	Pinia cart store with add/remove/update actions
<code>feature/auth</code>	Authentication store, login view, NavBar integration
<code>feature/ui-enhancements</code>	Visual polish and UI refinements

Development Timeline (Key Commits)

Phase	Commits	Description
Foundation	<code>init</code>	Vite + Vue 3 + TypeScript project scaffold
Core Features	<code>feat: search-filter</code> , <code>feat: router</code>	Client-side filtering, Vue Router, product detail page
State Mgmt	<code>feat: cart-logic</code> , <code>feat: auth</code>	Pinia stores for cart and authentication
UI Design	<code>feat(ui): redesign</code> , <code>feat(ui): landing page</code>	Premium UI redesign and five-section landing page
Dark Mode	<code>feat(ui): dark mode</code> , <code>feat(dark-mode): all components</code>	Full dark theme with <code>useTheme</code> composable
Polish	<code>fix: TypeScript build errors</code> , <code>feat(ui): refine navbar</code>	Build fixes, final UI refinements, deployment

5. Tools & Technologies

Category	Tool	Version
Framework	Vue 3 (Composition API)	3.5.24
Language	TypeScript	5.9.3
Build Tool	Vite	6.0.0
Styling	Tailwind CSS	3.4.17
State Management	Pinia	3.0.4
Routing	Vue Router	5.0.2
HTTP Client	Axios	1.13.4
Utilities	@vueuse/core	14.2.0
Package Manager	Bun	1.3.x
API	DummyJSON	—
Deployment	Netlify	—
Version Control	Git + GitHub	—
AI Tools	Gemini (Google AI Studio)	gemini-3-flash-preview

6. How to Run

```
# Prerequisites: Node.js 18+ or Bun 1.x
```

```
# 1. Install dependencies
```

```
npm install          # or: bun install
```

```
# 2. Start development server
```

```
npm run dev          # or: bun dev
```

```
# 3. Build for production
```

```
npm run build        # or: bun run build
```

4. Preview production build

npm run preview # or: bun preview